

Contents

Guide 11 — Security, the simple version	1
1. The idea, in three sentences	1
2. The measures	1
3. Where it comes from	2
4. The files	2
5. Jury questions — ready answers	2

Guide 11 — Security, the simple version

How Mobile Match Algeria defends against the common web risks — in plain words, with the files and ready answers for the jury.

1. The idea, in three sentences

Security is handled in **layers**, each guarding a different risk: inputs are **validated** before they are trusted, passwords are **hashed**, sensitive endpoints are **rate-limited**, and access is **controlled by role**. No single measure is relied on alone.

2. The measures

- **Input validation (zod).** The write endpoints (register, recommendations, profile, saved-offers) check every request body against a **zod schema** before doing anything, so malformed, oversized, or injection-style payloads are rejected up front. Read-only query parameters are parsed through typed guards.
 - **Password security (bcrypt).** Passwords are hashed with **bcrypt (cost 12)**, salted and slow by design; the plain password is never stored or logged. Login compares against the hash, never the raw value.
 - **Rate limiting.** A database-backed limiter (the `RateLimit` table) caps attempts per IP per time window — for example **register is limited to 5 attempts per 15 minutes** — to blunt brute-force and abuse. It **fails open** (a database hiccup never locks real users out).
 - **Access control (roles).** The user's `role` is carried in the **signed JWT** and checked **on the server** for admin endpoints, so a normal user cannot reach admin actions by guessing a URL. The two notification streams are role-filtered the same way.
 - **Safe database access.** Queries go through **Prisma**, which parameterises them (no string concatenation), so SQL injection is structurally avoided; the one raw full-text query uses bound parameters plus an input scrub.
 - **Output safety.** React escapes rendered values by default, which neutralises stored XSS.
 - **Ethical scraping.** Only public marketing pages are read, at a respectful rate, with the user agent identified and no personal data collected.
-

3. Where it comes from

These are the standard, OWASP-aligned web-security practices: **validate all input, hash passwords with a slow salted function (bcrypt), rate-limit sensitive endpoints, enforce authorization on the**

server, and parameterise database queries. Nothing here is bespoke cryptography — the project uses established, audited tools.

4. The files

- **lib/rate-limit.ts** — the per-IP, per-window limiter (fail-open).
 - **lib/auth.ts** — bcrypt password check, JWT with role.
 - **api/auth/register/route.ts** — zod schema + rate limit + bcrypt hash.
 - **api/admin/claim/route.ts** and the other admin endpoints — server-side role checks.
 - The Chapter 3 security table summarises every mechanism and the risk it covers.
-

5. Jury questions — ready answers

- **How are passwords protected?** bcrypt hash (cost 12), salted; never stored or logged in plain text.
 - **What about SQL injection?** Prisma parameterises every query; the single raw full-text query uses bound parameters and scrubs the input, so user text is never concatenated into SQL.
 - **What about brute-forcing the login or sign-up?** The rate limiter caps attempts per IP per window (e.g. 5 sign-ups / 15 min).
 - **Can a user reach admin endpoints?** No — the role is in the signed token and verified on the server for every admin route.
 - **What about XSS?** React escapes output by default, and inputs are validated with zod.
 - **Why does the rate limiter fail open?** So a transient database error degrades to “no limiting” rather than locking everyone out; availability is preserved while the normal case is still protected.
-

Companion to the authentication and testing guides.